Map Plugin — Mimari

X-Plane 11 için katmanlı harita eklentisi · sürüm 0.15 · kodun nasıl çalıştığı, hangi parçanın ne yaptığı ve parçaların birbirine nasıl bağlandığı

Bu belge kodu ilk kez okuyacak bir geliştirici için yazıldı. Kullanım ve test listesi `README.md` 'de; yapay zekâ asistanları için çalışma kuralları ve proje durumu `AGENTS.md` 'de.

1. Amaç ve temel fikir
2. Büyük resim
3. Bağımlılık kuralı: çekirdek ve X-Plane kabuğu
4. Bir karede ne oluyor?
5. Thread modeli
6. Koordinat sistemleri ve görünüm
7. Katman sistemi
8. Havalimanı verisi (apt.dat)
9. Zemin (DSF sahnesi)
10. Nav verisi, hava yolları, uçak
11. Çizim katmanı (OpenGL 3.3)
12. Arayüz ve fare
13. Harita modları (tema)
14. Ayar ve önbellek dosyaları
15. Dosya dizini
16. Testler ve araçlar
17. Hız ve bellek yöntemleri

1. Amaç ve temel fikir

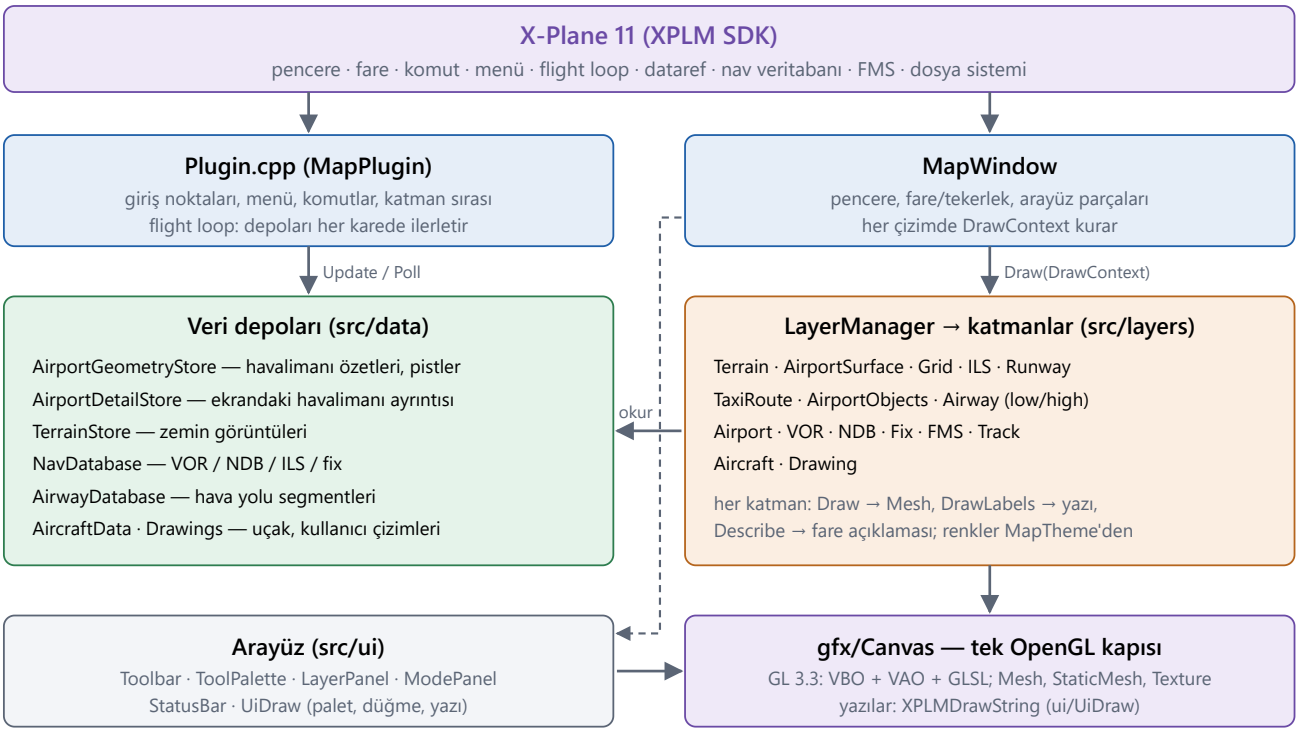
Eklenti, X-Plane'in içinde **kendi penceresinde** çalışan bir harita çizer. Dış harita servisi (OSM, karo sunucusu) kullanmaz; her şey o bilgisayarda kurulu X-Plane verisinden gelir:

- **Zemin:** kurulu DSF sahne dosyaları (arazi türü, su, kabartma).
- **Havalimanları:** apt.dat dosyaları (pistler, apron, taksi yolları, boya çizgileri, kapılar, ışıklar).
- **Navaid'ler:** X-Plane SDK'sının nav veritabanı (VOR, NDB, ILS, fix).
- **Hava yolları:** `earth_awy.dat` .
- **Uçak, FMS rotası:** dataref'ler ve FMS API'si.

Böylece haritada görünen her şey simülatördekiyle birebir aynıdır; yeni sahne kurulunca harita da kendiliğinden güncellenir. Asıl kullanım yeri simülatörlerin **eğitmen istasyonu (IOS)** ekranlarıdır.

Neden X-Plane'in kendi haritasına (`XPLMMap`) katman eklemiyoruz? O pencerenin yan bölmesi kaldırılamıyor, X-Plane'in kendi katmanları açılıp kapatılamıyor ve harita katmanları fare olayı almıyor. Kendi penceremizde bu sınırların hiçbiri yok.

2. Büyük resim



Şekil 1 — Ana parçalar. Düz oklar veri/çağrı yönü; kesikli çizgiler MapWindow'un arayüz parçalarını çizdirmesi.

Okuma sırası önerisi: `Plugin.cpp` → `MapWindow.cpp` (`Draw`) → `Layer.h` → basit bir katman (`layers/GridLayer.cpp`) → `gfx/Mesh.h` ve `gfx/Canvas.h` → ilgilendiğin veri deposu.

3. Bağımlılık kuralı: çekirdek ve X-Plane kabuğu

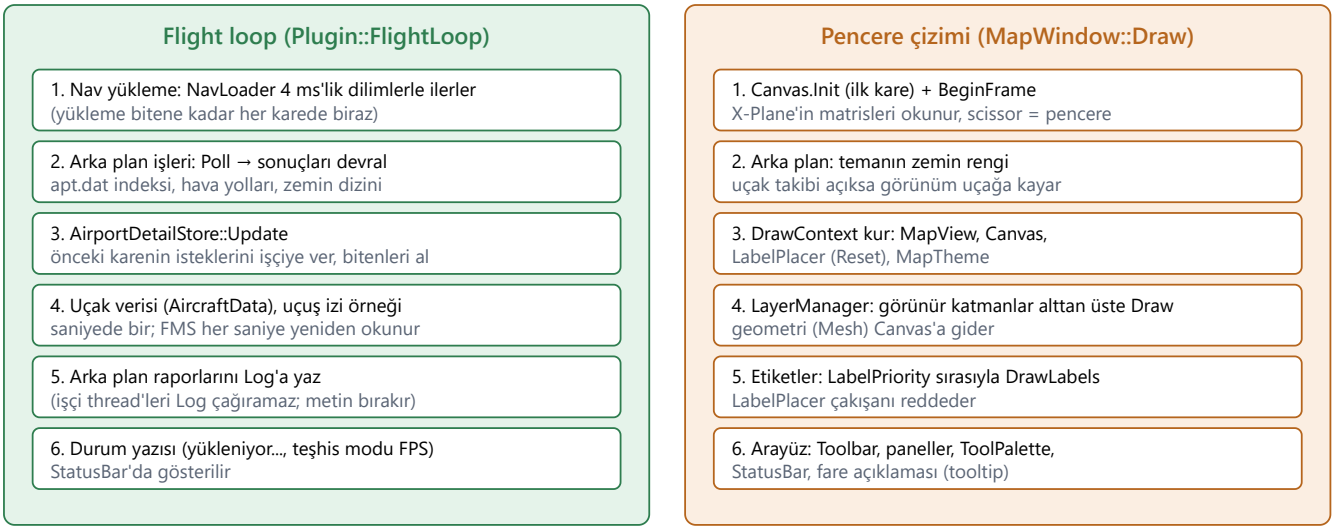
> **Kural:** kodun büyük kısmı X-Plane'e bağımlı **değildir** ("çekirdek"). XPLM başlığı yalnızca ince bir kabukta kullanılır. Bu sayede çekirdek, X-Plane açılmadan testlerde (`tests/`) ve önizleme aracında (`tools/preview`) çalışır.

Kabuk (XPLM kullanır)	Neden
`Plugin.cpp`	Eklenti giriş noktaları, menü, komut, flight loop
`MapWindow.*`	Pencere oluşturma, fare/tekerlek callback'leri, pop-out
`data/NavLoader.*`	SDK'nın nav veritabanını gezer
`data/AircraftData.*`	Dataref'ler (konum, yön)
`layers/FmsLayer.cpp`	FMS API'si
`Log.cpp`	`XPLMDebugString` → Log.txt
`gfx/Canvas.cpp` , `gfx/Texture.cpp`	Grafik durumu, doku numarası
`ui/UiDraw.cpp`	Yazı (`XPLMDrawString`)

CMake bu ayrımı iki listeyle yapar: `CORE_SOURCES` (eklenti + testler + önizleme) ve `PLUGIN_SOURCES` (sadece eklenti; önizleme XPLM fonksiyonlarını `tools/preview/XplmStubs` ile taklit eder). OpenGL çağrısı ise sadece `src/gfx/` içindedir.

4. Bir karede ne oluyor?

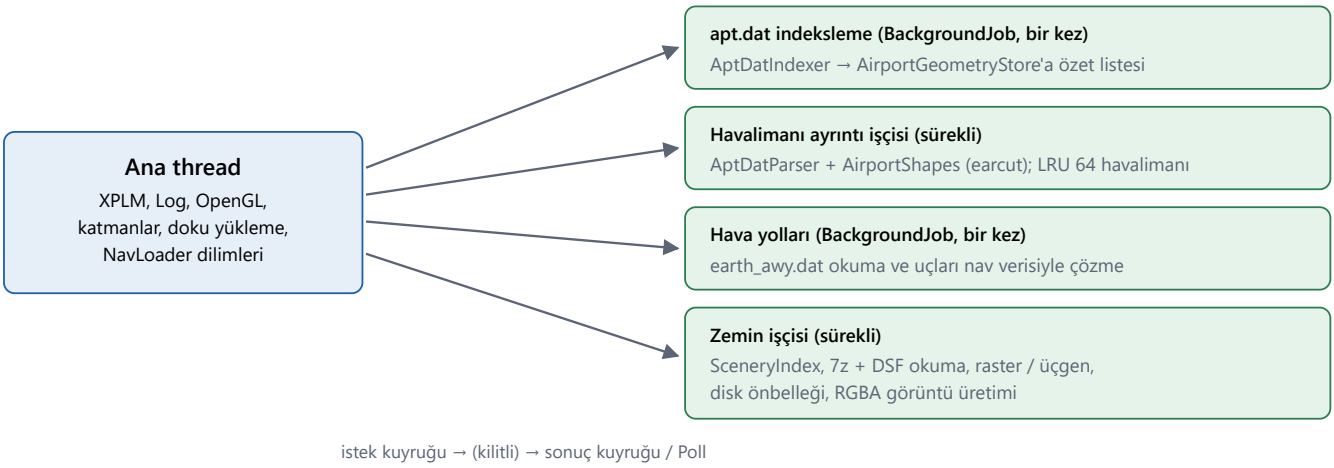
X-Plane eklentiye iki ayrı yerden zaman verir: **flight loop** (her simülasyon karesi) ve pencerenin **draw callback**'i (pencere çizilirken). İkisi de ana thread'de çalışır.



Şekil 2 — Bir karede ana thread'in yaptıkları.

Önemli tasarım noktası: katmanlar veriyi **istekle** alır. Bir katman ekrandaki bir havalimanının ayrıntısına ihtiyaç duyduğunda `AirportDetailStore::Use(i)` çağırır; ayrıntı hazırsa hemen döner, değilse istek kaydedilir ve katman o karede yer tutucu (yanıp sönen kesikli kutu) çizer. Bir sonraki flight loop isteği işçiye verir. Zemin de aynı biçimde çalışır (`TerrainStore::Request` / `PopImage`). Böylece hiçbir yavaş iş çizimi bekletmez.

5. Thread modeli



Şekil 3 — Arka plan thread'leri. Hepsi saf C++; X-Plane'i hiç çağırmaz.

> **Kurallar:** (1) Arka plan thread'inde hiçbir XPLM fonksiyonu, `Log` dahil, çağrılmaz; iş bir rapor metni döndürür, ana thread loglar. (2) Katmanlar sadece ana thread'de çalışır ve depoların ana thread tarafındaki kopyasını okur. (3) İşçiyle paylaşılan kuyruklar `std::mutex` + `condition_variable` ile korunur; tek seferlik işler `BackgroundJob::Poll` ile devralınır (join, yazılanların görünür olmasını garanti eder).

6. Koordinat sistemleri ve görünüm

Sistem	Birim	Nerede kullanılır
Coğrafi	lat/lon (derece)	Tüm veriler, çizimler, ayarlar
Mercator	0..1 (dünya bir kare)	`MapView` merkezi; havalimanı şekilleri referansa göre fark olarak
Boxel	XPLM pencere birimi (global masaüstü)	Bütün çizim ve fare; UI ölçeği %100 değilse pikselden farklı
Piksel	gerçek ekran pikseli	Sadece `glScissor`; dönüşüm `gfx/GlScreen`

`MapView` haritanın "kamerası"dır: Web Mercator, merkez + sürekli zoom (zoom 0'da dünya 256 boxel; her zoom adımında kenar iki katı; en çok 16.5). `ToScreen` / `ToLatLon` dönüşümleri ve imleç etrafında zoom (`ZoomAt`) buradadır. Web Mercator'da ekrana çevirme bir ölçek + kaydırma olduğu için, sabit şekiller GPU'ya bir kez yüklenip her karede sadece bu iki sayı değiştirilerek çizilebilir (`StaticMesh` , bölüm 11).

7. Katman sistemi

7.1 Layer arayüzü

```
class Layer                                  // src/Layer.h
{
    Draw(const DrawContext&)                 // geometri: Mesh uret, Canvas'a ver
    DrawLabels(const DrawContext&)           // yazilar; LabelPlacer'dan yer iste
    Describe(const DrawContext&, x, y)       // fare ustundeyse kisa aciklama (tooltip)
    LabelPriority()                          // kucuk = etiketi once yerlesir
    Id() / Name() / IsVisible() / DefaultVisible()
    AddSubLayer(id, name) / IsSubLayerVisible(i)   // istege bagli alt katmanlar
};

struct DrawContext
{
    const MapView* mView;        // projeksiyon
    Gfx::Canvas*   mCanvas;      // tek cizim kapisi
    LabelPlacer*   mLabels;      // ortak etiket / sembol cakisma onleyici
    const MapTheme* mTheme;      // harita modunun renkleri
    ...                          // pencere siniri, fare konumu
};
```

Etiket yerleşimi: `LabelPlacer` ekrana konan kutuları hatırlar; yeni kutu öncekilerden biriyle çakışıyorsa reddedilir. Katmanlar etiketlerini `LabelPriority` sırasıyla ister (havalimanı 10, pist 15, VOR 20, NDB 30, ILS 40, fix 60, hava yolu 70); semboller de `Reserve` ile kutularını önceden ayırır. Sonuç: hiçbir yazı bir sembolün üstüne binmez, sığmayan yazı yakınlaşınca görünür.

Alt katmanlar: bir katman yapıcısında parçalarını `AddSubLayer("pavement", "Pavement")` ile tanıtır, `Draw` içinde `IsSubLayerVisible(indeks)` ile o parçayı çizip çizmeyeceğine bakar. Panelde katmanın altında açılır liste olarak çıkar; ayar anahtarı `<katman>.<alt katman>`.

7.2 Çizim sırası ve katmanlar

Sıra `Plugin.cpp::CreateLayers()`'tadır (alttan üste). Zoom eşikleri her katmanın .cpp dosyasının başındaki sabitlerdedir.

#	Katman	Veri	Ne çizer / alt katmanlar
1	`TerrainLayer`	TerrainStore	Zemin dokuları (zoom 5+); seviye seçimi, doku LRU (512 MB)
2	`AirportSurfaceLayer`	AirportDetailStore	Apron/taksi yolu yüzeyi, kenarları, havalimanı sınırı, boya çizgileri
3	`GridLayer`	—	Enlem/boylam ızgarası, aralık zoom'a göre
4	`NavaidLayer` (ILS)	NavDatabase + pistler	Localizer tüyü pist eşiğinden (pistlerin altında kalsın diye burada)
5	`RunwayLayer`	AirportGeometryStore	Gerçek ölçekli pistler; kaplamalı/kaplamasız/su, blast pad, işaretler, numaralar
6	`TaxiRouteLayer`	AirportDetailStore	ATC taksi ağı (1201/1202), varsayılan kapalı
7	`AirportObjectsLayer`	AirportDetailStore	Işıklar, helipad, beacon, rüzgâr tulumu, kapı/bağlama/hangar, park adları, PAPI/VASI, tabelalar

8–9	`AirwayLayer` ×2	AirwayDatabase	Alçak / yüksek hava yolları, adlar, yön okları
10	`AirportLayer`	NavDatabase + özet	Havalimanı sembolleri ve adları (zoom'a göre seyreltme, kule rengi)
11–13	`NavaidLayer` (VOR, NDB, Fix)	NavDatabase	Semboller, kimlik/frekans; VFR'de VOR pusula gülü
14	`FmsLayer`	XPLM FMS	Uçuş planı (geçilmiş / aktif / kalan bacak)
15	`TrackLayer`	AircraftData	Uçuş izi
16	`AircraftLayer`	AircraftData	Uçak sembolü ve yön çizgisi
17	`DrawingLayer`	Drawings	Kullanıcı çizimleri: noktalar, rotalar, bölgeler, adlar/mesafeler

Ortak yardımcılar: `layers/MapSymbols` (sembol ölçüleri, etiketi sembolün yanına yerleştirme), `layers/AirportVisibility` (ekrana değen havalimanlarını haritanın merkezine yakınlık sırasıyla bulur — ayrıntısı önce merkezdekinin okunsun diye).

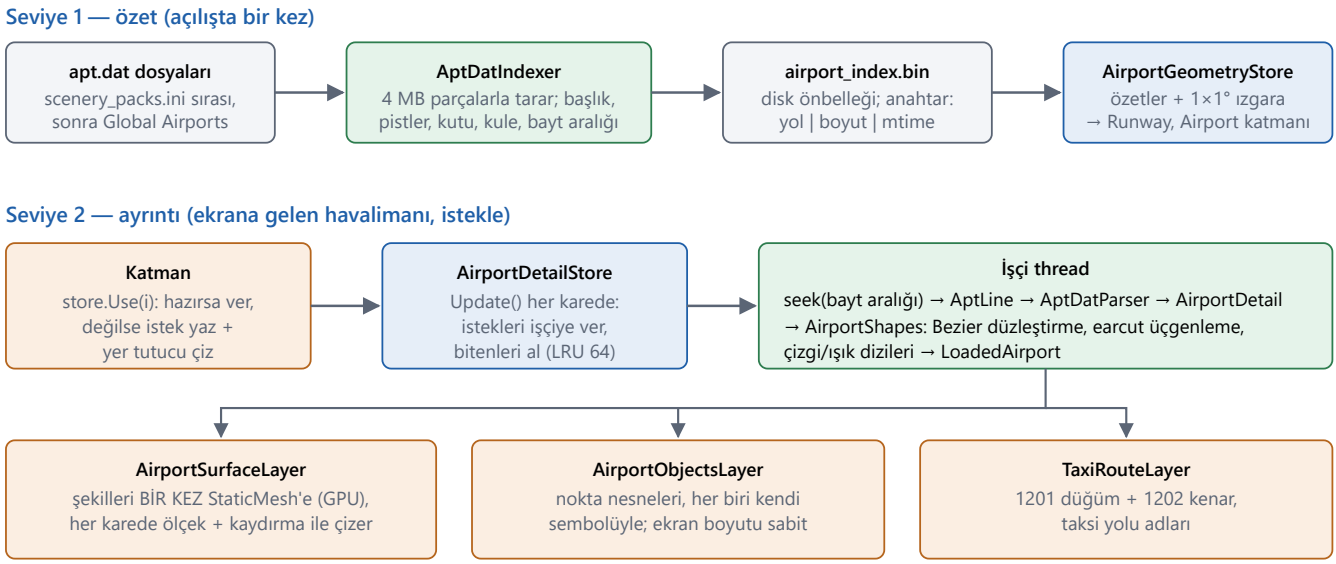
7.3 Yeni katman eklemek

1. `Layer` 'dan türeyen bir sınıf yaz (örnek: `layers/GridLayer.*`); `Draw` , gerekirse `DrawLabels` ve `Describe` .
2. Renkleri sabit yazma, `pContext.mTheme` 'den oku (gerekirse `MapTheme` 'ye alan ekle).
3. `Plugin.cpp::CreateLayers()` 'ta doğru sıraya ekle; dosyayı CMake listesine koy.
4. Mod başına varsayılan görünürlük: `MapTheme.cpp` (`mLayerDefaults`).
5. Panelde grubu: `ui/LayerPanel.cpp` içindeki `kGroups` tablosu.

Görünürlüğün ayar dosyasına kaydı, panelde onay kutusu ve alt katman listesi kendiliğinden gelir.

8. Havalimanı verisi (apt.dat)

apt.dat okuyucu projenin kendisinindir (saf C++, kaynak: XP-APT-1100 spesifikasyonu). Global Airports apt.dat'ı yüzlerce MB olduğu için iki seviyeli çalışır: **özet** her zaman bellekte, **ayrıntı** sadece ekrandaki havalimanları için.

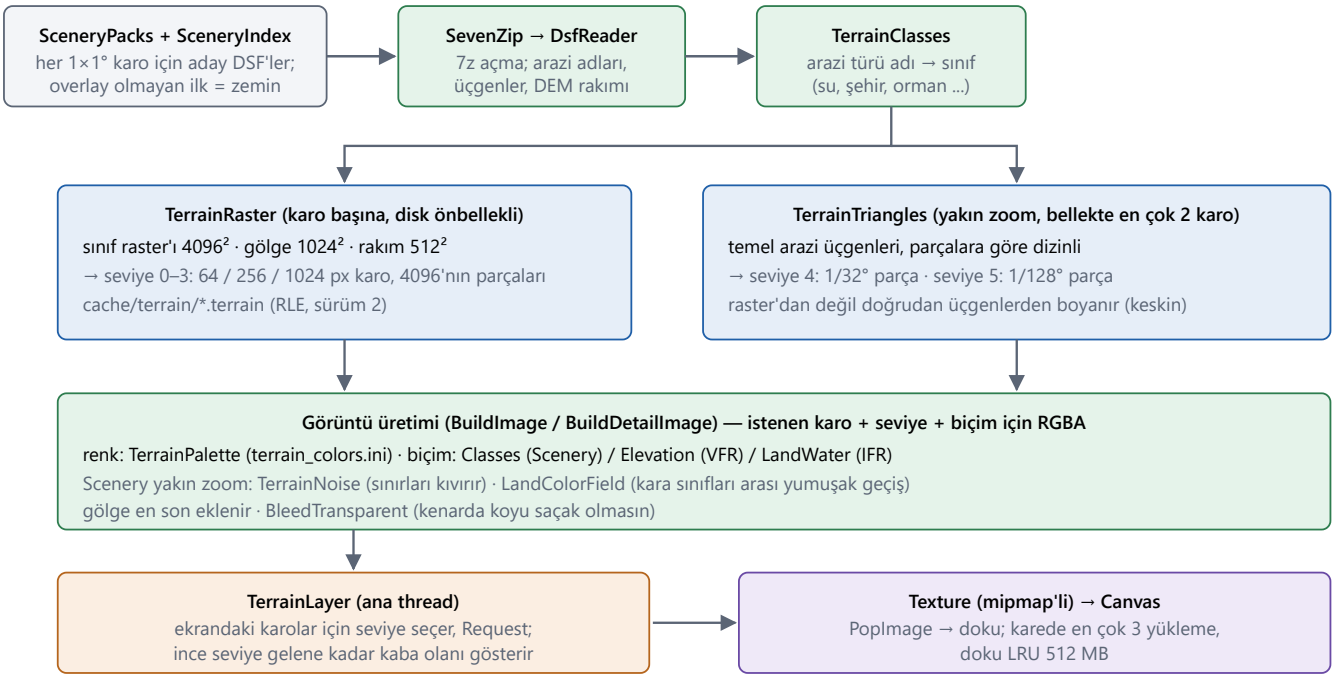


Şekil 4 — apt.dat'tan ekrana: iki seviyeli okuma.

- **Öncelik:** Custom Scenery paketleri `scenery_packs.ini` sırasıyla, sonra varsayılan apt.dat; aynı kimlikte ilk gelen kazanır (X-Plane'in kuralı). `scenery_packs.ini` sadece okunur.
- **AptLine:** sayıları yerel ayardan bağımsız okur (Türkçe Windows'ta `strtod` virgül bekleyebilir).
- **AirportDetail:** apt.dat satır kodlarının birebir karşılığı (100 pist, 102 helipad, 110 yüzey, 120 çizgi, 130 sınır, 111–116 düğümler, 14/15/18/19/20/21, 1201/1202, 1300...).
- **AirportShapes:** hesaplar havalimanı çevresinde metre cinsinden düz bir düzlemde (LocalFrame) yapılır; sonuç noktaları referans noktasına göre Mercator farkı (float) olarak saklanır.
- **Gerçek veri ≠ spesifikasyon:** kodda `GERCEK VERI` notları var (düğüm sonunda 0–3 stil değeri, başlıktaki kule alanı hep 0, Bezier kontrol noktasının yönü...). Biçim değiştirmeden önce bu notları oku.
- **LocalizerMatch:** SDK localizer'ın sadece anten yerini verir; ILS tüyü eşikten başlasın diye uyumlu pist ucu bulunur.

9. Zemin (DSF sahnesi)

Zemin, o bilgisayarda kurulu **her** sahneyle çalışır (bölgeye özel iş yok). Özel sahne global sahneyi ezer, sahne güncellenince harita da güncellenir. Tüm iş eklentinin içinde, arka plandaki zemin işçisinde yapılır.



Şekil 5 — Kurulu DSF sahnesinden zemin dokusuna.

9.1 Seviye seçimi

Seviye	Ne zaman (ekranda derece başına piksel)	Görüntü	Kaynak
0	< 100	64 px, tüm karo	sınıf raster'ı
1	< 400	256 px, tüm karo	sınıf raster'ı
2	< 1600	1024 px, tüm karo	sınıf raster'ı
3	< seviye 4 eşiği	4×4 parçadan biri, 1024 px	sınıf raster'ı
4	yakın	32×32 parçadan biri (1/32°), 1024 px	DSF üçgenleri
5	çok yakın	128×128 parçadan biri (1/128°), 1024 px	DSF üçgenleri

9.2 Önbellek ve senkron kalma

- Her karonun önbellek dosyası, üretildiği **bütün aday DSF'lerin** yol + boyut + değişiklik zamanı listesiyle etiketlidir. Sahne eklenir, çıkarılır, sırası değişir veya güncellenirse etiket tutmaz, karo yeniden üretilir.
- DSF'i olmayan karo: 10°'lik klasörü bir Global paketinde varsa **deniz**, yoksa **veri yok**. Sadece overlay DSF'i olan karo için `.nobase` işareti yazılır (deniz sayılır).
- Biçim değişirse `TerrainRaster.cpp`'deki `kFileVersion` artırılır; eski dosyalar kendiliğinden yeniden üretilir. Palet varsayılanları değişirse `TerrainPalette::kPaletteVersion` artırılır.

9.3 Yakın zoom görüntüsü neden ayrı?

Sınıf raster'ı derece başına 4096 pikseldir (enlem 47'de piksel başına ~27 × 18 m); yaklaşınca her piksel ekranda bir blok olur. Bu yüzden seviye 4–5'te görüntü doğrudan DSF üçgenlerinden boyanır. Ama DSF'te arazi türü üçgen

başına atandığı için düz renkle boyanınca üçgenler keskin bir mozaik gibi görünür. Bunu iki adım düzeltir:

- **TerrainNoise** ("domain warp"): sınıf okunurken bakılan nokta, enlem/boylama bağlı deterministik bir gürültüyle birkaç on metre kaydırılır; sınırlar kıvrılır. Gürültü coğrafi koordinattan üretildiği için her seviyede ve her parçada aynıdır (dikiş olmaz).
- **LandColorField**: sadece kara renkleri belirli bir yer mesafesinde (`[blend] meters`) normalize bulanıklıkla karıştırılır; su ve kıyı keskin kalır, gölge sonra eklenir. Doku kenar payıyla üretilip kırpılır, komşu parçalar arasında dikiş olmaz.

Ayar anahtarları (`[blend]` , `[warp]` , `[closeup]`) `terrain_colors.ini` 'nin başındaki açıklamada listelidir (dosyayı `Plugin::LoadTerrainColors` yazar).

9.4 Zemin görüntüsünde kullanılan yöntemler

Yöntem	Nerede	Ne işe yarar
Üçgenden doğrudan boyama	`TerrainRaster::BuildDetailImage` , `TerrainTriangles`	Seviye 4 (1/32°) ve 5 (1/128°, ~0,85 m/piksel) parçaları raster'dan büyütülmez, DSF üçgenlerinden yüksek çözünürlükte boyanır: yakında piksel blokları görünmez. Üçgenler parçalara göre dizinlidir, bir parça için sadece ona değen üçgenler gezilir; bellekte en çok 2 karonun üçgenleri tutulur.
Bikubik gölge	`ShadeAtSmooth` (Catmull-Rom)	Gölge raster'ı yakında çift doğrusal okununca elmas deseni yapıyordu; bikubik okuma yumuşak kabartma verir. Rakım çift doğrusal okunur.
Saydam piksel doldurma	`BleedTransparent`	Saydam piksellerin rengi en yakın opak komşudan doldurulur (alfa 0 kalır). Doğrusal süzme ve mipmap saydam pikselin siyahını kenara karıştırıp koyu saçak yapmasın diye.
Mipmap'li doku	`Texture::FromRgba(..., true)`	Uzaktan bakınca kumlanma / titreme olmaz (bellek 4/3 kat).
Sınır kıvırma	`TerrainNoise` , `WarpedClassAt` ; `[warp] meters / wavelength` (65 / 250)	Değer gürültüsü + fBm ile bakılan nokta kaydırılır; düz üçgen kenarları doğal sınırlara döner. Genlik pikselin ~0,75'inden küçükse (uzak seviyeler) kendiliğinden etkisizdir. Sadece Scenery (Classes) biçiminde.
Kara renklerinin yumuşak geçişi	`LandColorField` ; `[blend] meters` (150)	Normalize evrişim (kutu bulanıklığı): sadece kara örnekleri karışır, su rengine çekilmez, kıyı keskin kalır; gölge en son eklenir. Doku kenarlarda payla üretilip kırpılır, komşu parçalar arasında dikiş olmaz. Erim 1,5 pikselin altına düşünce plan boş döner (uzak seviyelerde eski görünüm değişmez).
Erim tavanı	`[blend] max_pixels` (5)	Yakın seviyelerde yumuşatma erimi doku pikseliyle sınırlanır; havalimanı ayrıntılarının göründüğü zoomlarda zemin bulanıklaşmaz.

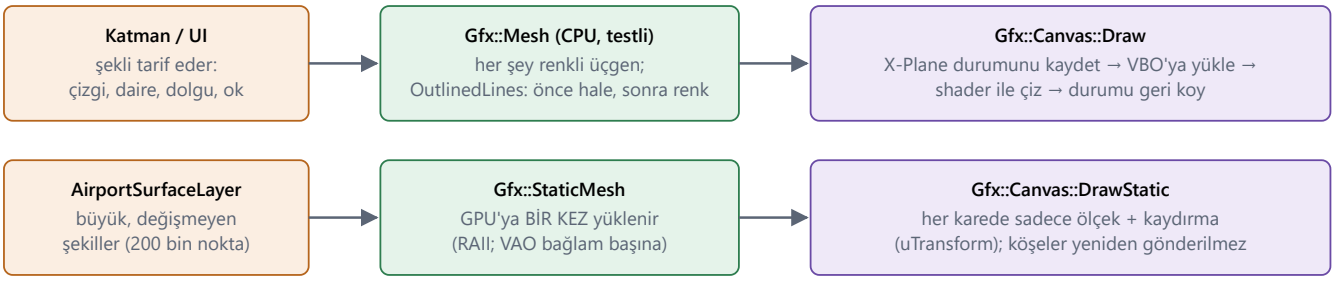
Yakın zoom sınıf karışımı	`CloseupMixAt`, `MixedColors`; `[closeup] class_mix` (0,3)	Piksel boyu 50 m'den küçüldükçe (5 m'de tam oran, arası logaritmik) kara sınıf renkleri ortak bir kara tonuna yaklaşır; yakında renk mozaiği sakinleşir. Su, kar ve "None" karışmaz.
Palet sürümü	`TerrainPalette::kPaletteVersion` (2)	Varsayılan renkler değişince dosyadaki eski renkler atılır, dosya yeni varsayılanlarla yeniden yazılır (shading / blend korunur).

10. Nav verisi, hava yolları, uçak

- **NavLoader** X-Plane'in o an yüklü nav veritabanını SDK ile gezer (`XPLMGetFirstNavAid` / `XPLMGetNextNavAid` / `XPLMGetNavAidInfo`). XPLM sadece ana thread'den çağrılabildiği için iş flight loop'ta 4 ms'lik dilimlere bölünür. Dosya okumaz: haritadaki navaid'ler simülatördekiyle aynıdır.
- **NavDatabase** kayıtları tutar ve her tür için ayrı bir **1×1° ızgara** (`GeoGrid`) kurar; katmanlar sadece ekrandaki hücrelere bakar (yüz binlerce fix var).
- **AirwayDatabase** `earth_awy.dat`'ı (XP11, 1100 biçimi) okur; önce `Custom Data`, sonra varsayılan. Satırda uçların koordinatı yok: kimlik + bölge + türle nav verisinden çözülür.
- **AircraftData** kullanıcı uçağının konumu ve yönünü dataref'lerden okur; uçak katmanı, iz ve "uçağı takip et" aynı veriyi kullanır.
- **Drawings** kullanıcı çizimlerinin modeli (lat/lon olarak saklanır, `map_drawings.txt`). `Geo` büyük daire mesafesi, ara nokta ve alan hesabını yapar.

SDK gerçekleri: ILS yönü **gerçek** yöndür (manyetik değil); TACAN frekansı kanal numarasıdır; "----" kimlikli yer tutucu ILS kayıtları filtrelenir. VOR pusula gülü `NavRecord::mVariation` ile döner.

11. Çizim katmanı (OpenGL 3.3)



Şekil 6 — İki çizim yolu: her karede üretilen Mesh ve GPU'da kalıcı StaticMesh.

Dosya	Görev
`gfx/Mesh.*`	CPU'daki renkli üçgen listesi. Modern GL'de `glLineWidth` güvenilmez olduğu için kalın çizgi, daire, çerçeve hepsi burada üçgene çevrilir; kalınlıklar boxel. `OutlinedLines` okunurluk için önce koyu/açık hale, sonra renkli çizgi.
`gfx/Canvas.*`	Tek çizim kapısı: VBO + VAO + GLSL 330. Her çizimde X-Plane'in program/VAO/VBO/cull durumunu kaydedip geri koyar; matris = GL_PROJECTION × GL_MODELVIEW (BeginFrame'de okunur). Dokulu şeritler (zemin) de buradan.
`gfx/StaticMesh.*`	GPU'da kalıcı şekil; `DrawStatic` ile ölçek + kaydırma.
`gfx/ContextVertexArray.*`	Pop-out pencere ayrı OpenGL bağlamında çizilir. Buffer, doku ve shader'lar bağlamlar arasında paylaşılır ama **VAO paylaşılmaz**; başka bağlamın VAO'sunu bağlamak X-Plane'i çökertir. Bu sınıf VAO'yu o anki bağlam için (gerekirse yeniden) oluşturur.
`gfx/GlApi.*`	Windows'un `opengl32.lib`'i sadece GL 1.1 verir; 3.3 fonksiyonları ilk çizimde `wglGetProcAddress` ile yüklenir. Bağlam değişimini (`ContextGeneration`) izler.
`gfx/Texture.*`	RAII doku; numara `XPLMGenerateTextureNumbers`, bağlama `XPLMBindTexture2d`. İsteğe bağlı mipmap.
`gfx/GlScreen.*`	Boxel → piksel dönüşümü (GL matrisinden) ve pencere alanıyla kırpma (scissor).

> **X-Plane ile birlikte yaşamak:** `XPLMDrawString` doku açar, X-Plane GL_CULL_FACE'i açık bırakabilir; bu yüzden her geometri çiziminden önce `XPLMSetGraphicsState` çağrılır ve cull kapatılır (Canvas bunu kendisi yapar — Canvas dışından çizim yapma). Yazılar sadece ASCII çizilebilir (Türkçe harf, derece işareti yok).

12. Arayüz ve fare

Pencerenin tamamı haritadır; kontroller haritanın üstünde ince ve yarı saydamdır. Arayüz parçaları **sadece çizer ve tıklamanın neye denk geldiğini söyler** (`HitTest`); ne yapılacağına `MapWindow` karar verir. Ortak palet ve çizim yardımcıları `ui/UiDraw`'dadır (`Ui::k...` renkleri).

Parça	Ne
`Toolbar`	Üst şerit: Layers · Map: <mod> | Follow / Center on aircraft | Draw tools … Pop out · Hide. Gruplar arasında ayırıcı; dar pencerede pencere düğmeleri ikinci satıra iner.
`ToolPalette`	Yüzen çizim paleti: Pan / Point / Route / Region + Finish / Undo / Clear. Başlıktan sürüklenir, seçili araca katlanır; yeri sağ üst köşeye göre `[tools]`'a kaydedilir.

`LayerPanel`	Grup › katman › alt katman listesi; üç durumlu kutu (hepsi / hiçbiri / karışık), ok ile aç/kapa, tekerlekle kaydırma. Gruplar `kGroups` tablosunda.
`ModePanel`	Harita modu seçimi (tek seçim).
`StatusBar`	İmleç koordinatı, zoom, durum mesajı, ölçek çubuğu (NM).

Fare akışı (MapWindow)

```
basildi  -> once kontroller: HandleControlClick (toolbar, paneller, palet basligi)
            degilse haritaya basildi (mPressOnMap)
hareket  -> 4 px'ten fazla: surukleme = haritayi kaydir (ucak takibi kapanir)
            palet basligindan tutulduysa: paleti tasi
birakti  -> hic hareket yoksa tiklama: HandleMapClick
            Point / Route / Region modunda nokta / kose ekler
sag tik  -> yarim rota / bolgeyi bitirir
tekerlek -> imlecin altindaki nokta sabit kalarak zoom (takipteyken ucagin etrafinda);
            imlec katman panelindeyse paneli kaydirir
```

Tıklama asla alttaki pencereye geçirilmez (SDK dokümanına göre fare takibini bozar). Pop-out modunda imleç konumu cursor callback'ten okunur.

13. Harita modları (tema)

`src/MapTheme.*` dört modu tanımlar. Her mod bir `MapTheme`: arka plan ve su rengi, zemin biçimi, hale rengi, açık/koyu tema bayrağı, ızgara, havalimanı/navaid/hava yolu/pist renkleri ve mod başına varsayılan katman görünürlüğü. `DrawContext::mTheme` her zaman doludur; katmanlar renkleri buradan okur.

Mod	Zemin biçimi	Özellik
Scenery	Classes — arazi sınıfları + kabartma	Koyu tema
VFR	Elevation — rakım rampası + kabartma	Kuleli havalimanı mavi, kulesiz macenta; VOR pusula gülü
IFR Low	LandWater — iki düz renk	Açık tema; alçak hava yolları, VOR/NDB/ILS/fix
IFR High	LandWater	Sadece büyük havalimanları, yüksek hava yolları

Mod değişince (`MapWindow::SetMapMode`) eski modun katman görünürlüğü kaydedilir, yenisininki yüklenir. Zemin görüntü anahtarı biçimi de içerir (`TerrainImageKey::mStyle`), böylece her modun dokusu ayrı üretilip önbelleklenir.

14. Ayar ve önbellek dosyaları

Hepsi `C:\X-Plane 11\Resources\plugins\MapPlugin\` altında (eklenti X-Plane klasöründe başka hiçbir yere yazmaz). INI dosyaları `Settings` sınıfıyla okunur/yazılır.

Dosya	İçerik
`64/win.xpl`	Eklentinin kendisi
`map_settings.ini`	`[window]` yer/boyut/görünür · `[view]` merkez, zoom, takip, kontroller, `mode` · `[tools]` palet · `[layers.scenery]`, `[layers.vfr]`, `[layers.ifr_low]`, `[layers.ifr_high]` (katman ve `katman.alt` anahtarları); eski `[layers]` Scenery'ye okunur
`map_drawings.txt`	Kullanıcı çizimleri (metin)

`terrain_colors.ini`	`[colors]`, `[shading]`, `[blend]`, `[warp]`, `[closeup]`, `[palette]` `version`
`cache/terrain/`	Zemin raster önbelleği (silinebilir)
`cache/airports/airport_index.bin`	apt.dat özet indeksi (silinebilir)

15. Dosya dizini

src/ — giriş, pencere, çekirdek

Dosya	Görev
`Plugin.cpp`	XPluginStart/Stop/Enable/Disable; menü "Map Plugin"; komutlar `ozdev/map/toggle`, `controls`, `diagnostics`, `next_mode`; flight loop; `CreateLayers()`; terrain_colors.ini
`MapWindow.*`	Pencere (pop-out), fare/tekerlek, arayüz olayları, mod değişimi, ayar kaydı, tooltip
`MapView.*`	Web Mercator kamera, lat/lon ↔ boxel, imleç etrafında zoom
`ScreenMath.*`	Ekran koordinatı yardımcıları (dikdörtgen, nokta)
`Layer.h`, `LayerManager.*`	Katman arayüzü, alt katmanlar, DrawContext; liste, aç/kapa, mod başına kayıt
`MapTheme.*`	Harita modları ve renkleri
`LabelPlacer.*`	Etiket/sembol çakışma önleme
`MapFormat.*`	Koordinat/mesafe yazı biçimleri, ızgara ve ölçek aralığı
`Geo.*`, `GeoGrid.*`	Büyük daire hesapları, poligon üçgenleme; 1×1° uzamsal indeks
`Drawings.*`	Kullanıcı çizim modeli ve dosyası
`Settings.*`, `Log.*`, `BackgroundJob.*`, `Color.h`	INI, Log.txt yazımı, tek seferlik arka plan işi, Rgba

src/data/ — veri

Dosya	Görev
`AptLine.*`	apt.dat satırını yerel ayardan bağımsız okuyan alan okuyucu
`AptDatIndexer.*`	Özet indeksi + disk önbelleği
`AptDatParser.*`	Tek havalimanı metni → `AirportDetail`
`AirportGeometry.*`, `AirportGeometryStore.*`	Özet yapısı; özet deposu + ızgara
`AirportDetail.h`, `AirportDetailStore.*`	Ayrıntı yapısı; istekle okuyan işçi + LRU
`AirportShapes.*`	Bezier, earcut üçgenleme, çizgi/ışık dizileri
`LocalizerMatch.*`	ILS → pist eşiği
`NavDatabase.*`, `NavLoader.*`	Nav kayıtları + ızgara; SDK'dan dilimli yükleme
`AirwayDatabase.*`	earth_awy.dat
`AircraftData.*`	Uçak konumu/yönü
`SceneryPacks.*`, `SceneryIndex.*`	Sahne sırası; karo → aday DSF'ler
`SevenZip.*`, `DsfReader.*`	7z açma; DSF okuma (arazi adları, üçgenler — yama bayrağı ve düzlem sayısı dahil, DEM)
`TerrainClasses.*`	Arazi türü → sınıf, palet (sürüm 2), rakım rampası (`ElevationColor`), blend / warp / closeup ayarları

`TerrainRaster.*`	Sınıf/gölge/rakım raster'ları, görüntü üretimi (kaba ve yakın), önbellek dosyası
`TerrainTriangles.*`	Yakın zoom için hafif üçgen deposu (parça dizinli)
`TerrainNoise.*` , `LandColorField.*`	Sınır kıvırma gürültüsü; kara renklerinin yumuşak geçişi
`TerrainStore.*`	Zemin işçi thread'i, istek/sonuç kuyrukları, üçgen LRU

src/layers/, src/gfx/, src/ui/

Bölüm 7.2 (katmanlar), 11 (gfx) ve 12 (ui) tablolarına bak. Her dosyanın başında Türkçe bir açıklama bloğu vardır (ne yapar, neden, zoom eşikleri, şema); ayrıntı için önce o bloğu oku.

Diğer

Yer	Ne
`tests/tests.cpp`	X-Plane'siz testler (bölüm 16)
`tools/preview/`	`map_preview` : aynı katman kodunu XPLM taklitleriyle resme çizer
`tools/dsfinfo/`	`dsf_info` : bir DSF dosyasının içeriğini döker
`third_party/`	XPSDK (XPLM200..301 açık), LZMA SDK 23.01, mapbox earcut 2.2.4 (tek belgelenmiş yama: `README.txt`). Dış kod sadece burada.
`CMakeLists.txt`	Hedefler: `MapPlugin` (win.xpl), `map_tests` , `map_preview` , `dsf_info` , `lzma_sdk`

16. Testler ve araçlar

- **`map_tests`** (`-DMAP_TESTS=ON`): çekirdeği X-Plane'siz dener — Mesh geometrisi, MapView gidiş-dönüş ve zoom, LabelPlacer, ayarlar, Drawings, Geo, apt.dat ayrıştırma ve Bezier, AirportShapes, AirwayDatabase satırları, DSF okuma, zemin raster'ı, gürültü ve renk karışımı, temalar, mod başına görünürlük. Hepsi geçmeli.
- **`map_preview`** (`-DMAP_PREVIEW=ON`): eklentinin kendi katman ve arayüz kodunu gerçek X-Plane verisiyle görünmez bir OpenGL penceresinde çizip PPM resim yazar. Örnek: `map_preview --lat 47.45 --lon -122.30 --zoom 12 --mode vfr --out ksea.ppm` (ayrıca `--size --show --hide --panel --describe x,y --demo-drawings --no-ui`). Görsel kontrol için hızlıdır ama gerçek X-Plane denemesinin yerine geçmez.
- **`dsf_info`** : DSF'in özellikleri, arazi türleri, üçgen sayıları, yama bayrağı × düzlem sayısı; `--raw` ile açılmış hâlini yazar. DSF biçimiyle ilgili bir şey değiştirmeden önce gerçek dosyaya bununla bak.

17. Hız ve bellek yöntemleri (genel)

Haritanın X-Plane'in FPS'ini düşürmemesi için kullanılan yöntemler; yeni kod yazarken aynı kalıpları izle.

Yöntem	Nerede	Neden
1×1° uzamsal ızgara	`GeoGrid` , `NavDatabase` , `AirportGeometryStore`	Yüz binlerce kayıt var; her karede sadece ekrandaki hücrelere bakılır.
Dilimli yükleme	`NavLoader`	SDK sadece ana thread'den çağrılabildiği için iş flight loop'ta 4 ms'lik dilimlere bölünür; simülatör takılmaz.
İstekle, arka planda hazırlama	`AirportDetailStore` , `TerrainStore`	Katman `Use` / `Request` ile ister, hazır değilse yer tutucu çizer; yavaş iş çizimi bekletmez. En son istenen önce yapılır.

İki seviyeli veri	apt.dat özet + ayrıntı	Yüzlerce MB'lık dosyanın sadece özeti bellekte; ayrıntı ekrana gelen havalimanı için dosyanın o bayt aralığından okunur.
Disk önbellekleri	`airport_index.bin`, `cache/terrain`	Pahalı tarama / DSF açma bir kez yapılır; anahtar yol|boyut|mtime olduğu için kaynak değişince kendiliğinden yenilenir.
LRU sınırları	Ayrıntı (64 havalimanı), zemin dokusu (512 MB), üçgen (2 karo), raster	Bellek dolaşılan alanla sınırsız büyümez; en uzun süredir kullanılmayan atılır.
GPU'da kalıcı şekil	`StaticMesh` + `DrawStatic`	Büyük havalimanı yüzeyleri (yüz binlerce nokta) bir kez yüklenir; her karede sadece ölçek + kaydırma gönderilir.
Doku yükleme sınırı	`TerrainLayer`	Karede en çok 3 doku yüklenir (takılma olmasın); ince seviye gelene kadar elde olan kaba seviye gösterilir.
Merkezden dışa öncelik	`AirportVisibility`	Ekranın ortasındaki havalimanının ayrıntısı önce istenir.
Zoom eşikleri	Her katmanın .cpp başındaki `k...MinZoom` sabitleri	Uzakta anlamsız ayrıntı (fix, ışık, tabela, boya çizgisi) hiç işlenmez.
Hücreli çakışma testi	`LabelPlacer` (32 boxel hücreler)	Yeni etiket kutusu sadece değdiği hücrelerdeki kutularla karşılaştırılır.
Üçgenleme hızı	`third_party/earcut` (tek yama)	Earcut'ın `isValidDiagonal`'ında ucuz yerel testler pahalı halka taramasından önce yapılır; sonuç aynı, çok delikli apronlarda çok daha hızlı. Ayrıntı `third_party/earcut/README.txt`.
Tek çizim kapısı, toplu çizim	`Canvas`, `Mesh`	Bir katmanın bütün sembolleri tek Mesh'te toplanıp tek çağrıda çizilir.